FINALEMAILTEST.ino

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <DHT.h>
#include "Base64.h"

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// WiFi Credentials
const char* ssid = "Redacted for Privacy";
const char* password = "Redacted for Privacy";

// Gmail SMTP Server
const char* smtp_server = "smtp.gmail.com";
const int smtp_port = 465;

// Email Auth
const char* sender_email = "Redacted for Privacy@gmail.com";
const char* app_password = "Redacted for Privacy";

// Email Recipients
const char* recipient_email1 = "Redacted for Privacy@clarkparkgarden.org";
const char* recipient_email2 = "Redacted for Privacy@gmail.com";

// Sleep config
#define uS_TO_S_FACTOR 1000000ULL
#define TIME_TO_SLEEP 86400  // 1 day

void setup() {
  Serial.begin(115200);
  dht.begin();

  Serial.println("Connecting to WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected!");

  float tempF = dht.readTemperature(true);
  float humidity = dht.readHumidity();

  if (isnan(tempF) || isnan(humidity)) {
    Serial.println("Sensor read failed. Sleeping...");
    goToSleep();
  }

```

```
  tempF -= 5;  // Calibration adjustment

  String status = analyzeCompost(tempF, humidity);

  String message = "=== Composter 1 Report ===\n\n";
  message += "Temp: " + String(tempF, 1) + "F\n";
  message += "Humidity: " + String(humidity, 1) + "%\n";
  message += "Status: " + status;

  Serial.println(message);

  sendEmail("Daily Compost Status - Composter 1", message, recipient_email1);
  sendEmail("Daily Compost Status - Composter 1", message, recipient_email2);

  goToSleep();
}

void loop() {
  // Not used
}

String analyzeCompost(float temp, float hum) {
  String tempStatus, humStatus;

  if (temp < 90)
    tempStatus = "Too Cold";
  else if (temp <= 160)
    tempStatus = "Optimal";
  else
    tempStatus = "Too Hot";

  if (hum < 40)
    humStatus = "Too Dry";
  else if (hum <= 60)
    humStatus = "Ideal Moisture";
  else
    humStatus = "Too Wet";

  return tempStatus + " & " + humStatus;
}

void sendEmail(const char* subject, String messageBody, const char* recipient_email) {
  WiFiClientSecure client;
  client.setInsecure();

  Serial.println("Connecting to SMTP server...");
  if (!client.connect(smtp_server, smtp_port)) {
    Serial.println("SMTP connection failed!");
    return;
```

```
  }

  client.println("EHLO esp32");
  delay(500);
  client.println("AUTH LOGIN");
  delay(500);
  client.println(base64::encode(String(sender_email)));
  delay(500);
  client.println(base64::encode(String(app_password)));
  delay(500);

  client.println("MAIL FROM:<" + String(sender_email) + ">");
  delay(500);
  client.println("RCPT TO:<" + String(recipient_email) + ">");
  delay(500);
  client.println("DATA");
  delay(500);

  client.println("From: <" + String(sender_email) + ">");
  client.println("To: <" + String(recipient_email) + ">");
  client.println("Subject: " + String(subject));
  client.println("Content-Type: text/plain; charset=utf-8");
  client.println();
  client.println(messageBody);
  client.println(".");
  delay(500);

  client.println("QUIT");
  delay(500);

  Serial.println("Email sent to: " + String(recipient_email));
}

void goToSleep() {
  Serial.println("Disconnecting WiFi and entering deep sleep...");
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
  btStop();

  Serial.flush();
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
  esp_deep_sleep_start();
}
```